Universidad de Valladolid

Facultad de Ingeniería Informática

Evaluación y Rendimiento de Sistemas Software

Segunda Práctica: Pruebas de carga

Grupo: 14

Autores:
Martín Carmona, Mario
Martínez Vázquez, Elena
Rodrigues Álvarez, Emily Fabiana

Índice

1. Introducción **2**

2. Estructura del proyecto **3**

3. Diseño de la web **5**
3.1. Implementación de la página web 5

4. Pruebas de carga con Locust **9**
4.1. Actividades realizadas 9
4.2. Archivos para las pruebas de carga 9
4.2.1. Archivo `load_test.py` 9
4.3. Resultados 12
4.3.1. Prueba 1 12
4.3.2. Prueba 2 13
4.3.3. Prueba 3 14
4.3.4. Prueba 4 15
4.4. Conclusiones 16

5. Monitorización con Grafana **18**
5.1. Archivos relevantes para la monitorización 18
5.1.1. Archivo `main.py` 19
5.1.2. Archivo `metrics.py` 20
5.1.3. Archivo `db.py` 22
5.2. Métricas registradas 23
5.2.1. Carga instantánea de CPU 23
5.2.2. Frecuencia instantánea de CPU 25
5.2.3. Uso de memoria RAM y memoria SWAP 28
5.2.4. Espacio en disco 32
5.2.5. Operaciones de entrada/salida (E/S) 34
5.2.6. Estadísticas de red: bytes enviados y recibidos 36
5.2.7. Conexiones de red activas 41
5.2.8. Número de procesos en ejecución 43
5.2.9. Lista detallada de los procesos en ejecución 47
5.3. Resultados 50

6. Conclusiones **52**

1. Introducción

En esta práctica se ha desarrollado un entorno completo para evaluar el rendimiento de una aplicación web simulando múltiples usuarios concurrentes.

El propósito principal ha sido analizar el comportamiento del sistema bajo carga, detectando posibles problemas y evaluando la repercusión en los recursos del sistema. Para realizarlo, hemos creado una tienda online sencilla usando Flask y una base de datos MySQL, la cual permite realizar operaciones como el registro de usuarios, navegación de productos, gestión del carrito y realización de pedidos. Además, hemos diseñado y ejecutado pruebas de carga con Locust, generando una simulación de distintos flujos de usuario.

Por otro lado, hemos incorporado un sistema de monitorización utilizando Grafana, que permite recoger las distintas métricas del sistema como el uso de CPU, memoria, disco, red, entre otras. Esta monitorización nos ha servido para poder realizar la evaluación del impacto real que han tenido las pruebas sobre nuestro entorno.

2. Estructura del proyecto

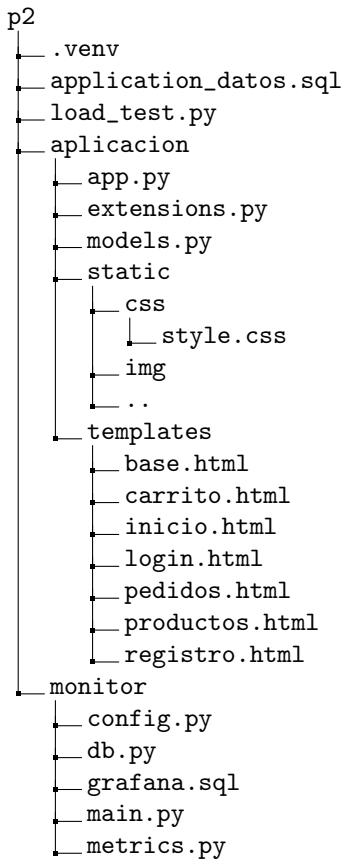
El proyecto se ha dividido en varios módulos independientes con el objetivo de facilitar la organización, la reutilización del código y la separación de responsabilidades. La estructura queda organizada de la siguiente manera:

- La carpeta `aplicacion` contiene toda la lógica de la página web, incluyendo los archivos Python del backend (`app.py`, `models.py`, etc.), las plantillas HTML del frontend y los recursos estáticos como CSS e imágenes PNG.
- La subcarpeta `monitor` incluye los scripts relacionados con la monitorización del sistema: desde el archivo SQL que define la estructura de la base de datos usada por Grafana, hasta los scripts Python que recolectan y almacenan las métricas de rendimiento.
- El archivo `load_test.py` define los flujos de usuarios simulados en las pruebas de carga utilizando Locust.
- Se ha utilizado un entorno virtual `venv` para aislar las dependencias de Python requeridas tanto por la página web como por los scripts de monitorización, facilitando su instalación y portabilidad. Aunque en el archivo .zip entregado no aparezca debido a temas de tamaño para la entrega, fue debidamente instalado y usado a lo largo del desarrollo de la práctica.

Por claridad y mantenimiento, se optó por trabajar con dos bases de datos independientes:

- `application_datos.sql`: contiene las tablas de la tienda online (usuarios, productos, pedidos, etc.).
- `grafana.sql`: define la base de datos que almacena las métricas del sistema que serán posteriormente visualizadas con Grafana.

A continuación, se muestra la estructura de carpetas y archivos del proyecto:


p2
.venv
application_datos.sql
load_test.py
aplicacion
app.py
extensions.py
models.py
static
css
style.css
img
..
templates
base.html
carrito.html
inicio.html
login.html
pedidos.html
productos.html
registro.html
monitor
config.py
db.py
grafana.sql
main.py
metrics.py

3. Diseño de la web

El sistema web fue diseñado como una aplicación cliente-servidor basada en la arquitectura MVC (Modelo-Vista-Controlador), utilizando tecnologías ligeras y ampliamente usadas en desarrollo web.

El backend está desarrollado en **Flask** (Python), gestionando rutas, sesiones de usuario, lógica de negocio y una API REST. Para el acceso a la base de datos se emplea el ORM **SQLAlchemy**.

La base de datos utilizada almacena la información de usuarios, productos, pedidos y métricas del sistema.

El frontend está construido con **HTML5**, **CSS3** y plantillas **Jinja2** integradas en Flask, con una interfaz sencilla que permite el registro, autenticación, navegación por productos y realización de pedidos.

La autenticación de usuarios se implementó utilizando hashing seguro de contraseñas con la biblioteca **Werkzeug**.

Además, se desarrollaron varios endpoints REST que permiten la interacción con la tienda mediante scripts automatizados. Estos endpoints facilitan la simulación de carga y evaluación del rendimiento con **Locust**, permitiendo emular patrones reales de usuarios como registro, login, consulta de productos y realización de pedidos.

3.1. Implementación de la página web

Fichero `aplicacion_datos.sql`

Este archivo contiene las tablas y datos para la tienda online, incluyendo usuarios, productos, carrito y pedidos:

- `usuarios`. Contiene la información de los usuarios registrados en la tienda.
- `productos`. Almacena el catálogo de productos disponibles.
- `carrito`. Representa los productos que un usuario ha añadido a su carrito de compras. Se asocia a la vez a un usuario y a un producto.
- `pedidos`. Registra las órdenes de compra hechas por los usuarios.
- `pedidos_item`.Tabla que representa los productos contenidos en cada pedido. Relación muchos-a-muchos entre pedidos y productos, con campos adicionales.

```
1 -- Creaci n de la base de datos
2 CREATE DATABASE IF NOT EXISTS tienda_online;
3 USE tienda_online;
4
5 -- Tabla de usuarios
6 CREATE TABLE IF NOT EXISTS usuarios (
7     id INT PRIMARY KEY AUTO_INCREMENT,
8     nombre VARCHAR(50) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
    contrase a_hash VARCHAR(255) NOT NULL,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla de productos
CREATE TABLE IF NOT EXISTS productos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    descripcion TEXT,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON
        UPDATE CURRENT_TIMESTAMP,
    img VARCHAR(200)
);

-- Tabla del carrito
CREATE TABLE IF NOT EXISTS carrito (
    id INT PRIMARY KEY AUTO_INCREMENT,
    usuario_id INT NOT NULL,
    producto_id INT NOT NULL,
    cantidad INT NOT NULL DEFAULT 1,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    FOREIGN KEY (producto_id) REFERENCES productos(id)
);

-- Tabla de pedidos
CREATE TABLE IF NOT EXISTS pedidos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    usuario_id INT NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    estado ENUM('pendiente', 'completado', 'cancelado')
        DEFAULT 'pendiente',
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);

-- Tabla de detalles del pedido
CREATE TABLE IF NOT EXISTS pedido_items (
    id INT PRIMARY KEY AUTO_INCREMENT,
    pedido_id INT NOT NULL,
    producto_id INT NOT NULL,
    cantidad INT NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id),
    FOREIGN KEY (producto_id) REFERENCES productos(id)
);

```

```sql
-- Datos de ejemplo para productos (5 art culos)
INSERT INTO productos (nombre, descripcion, precio, stock, img) VALUES
('Camiseta', 'Camiseta de algodon 100% color blanco', 15.99, 100, '/aplicacion/static/img/cami.png'),
('Pantalon', 'Pantalon jeans azul talla unica', 34.50, 50, 'aplicacion/static/img/pant.png'),
('Zapatillas', 'Zapatillas running talla 42', 49.95, 30, 'aplicacion/static/img/shoes.png'),
('Gorra', 'Gorra color roja', 12.75, 80, 'aplicacion/static/img/cap.png'),
('Sudadera', 'Sudadera de algodon', 20.99, 200, 'aplicacion/static/img/hoodie.png');
```

Listing 1: Contenido de aplicacion_datos.sql

Fichero `app.py`

Contiene la lógica principal de la aplicación web desarrollada con Flask. Define rutas protegidas, gestión de sesiones, carrito de la compra, pedidos y una API REST que será usada por Locust y otras herramientas externas.

A continuación se muestran los fragmentos más relevantes.

Configuración de Flask y SQLAlchemy

```python
app = Flask(__name__)
app.secret_key = '...'
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+mysqlconnector://...'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db.init_app(app)
```

Listing 2: Configuracón inicial de la aplicación

Middleware de sesión de usuario

```python
@app.before_request
def cargar_usuario():
    if 'usuario_id' in session:
        usuario = Usuario.query.get(session['usuario_id'])
        ...
```

Listing 3: Carga automática de usuario en cada request

Ruta de autenticación

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    ...
```

Listing 4: Inicio de sesión de usuario

Ruta de pedido

```
@app.route('/pedido/realizar')
@login_required
def realizar_pedido():
    ...
```

Listing 5: Creación de un pedido a partir del carrito

API REST

```
@app.route('/api/productos')
def api_productos():
    ...
```

Listing 6: Listado de productos vía API REST

4. Pruebas de carga con Locust

4.1. Actividades realizadas

Para la evaluación de rendimiento de la aplicación web se instaló y configuró la herramienta **Locust**, que permite simular usuarios virtuales (clientes) realizando diferentes acciones sobre el sistema.

Locust se ejecutó fuera de la máquina virtual donde se desplegaba la tienda online, con el fin de no contaminar los datos de las métricas de rendimiento recogidas, lo cual es la opción más lógica en este contexto, ya que permite aislar la carga generada por las pruebas del entorno evaluado.

Se diseñaron varios escenarios de prueba representativos del uso real de la tienda online, basados en la definición de clases de usuarios virtuales con distintas tareas y pesos:

- **Navegación por productos**: los usuarios consultan las páginas principales, la lista de productos y la API REST correspondiente.
- **Flujo completo de compra**: los usuarios seleccionan productos aleatorios, los añaden al carrito, visualizan el carrito y, con cierta probabilidad, realizan el pedido y consultan sus compras.
- **Registro y login de usuarios**: algunos usuarios nuevos se registran con datos generados aleatoriamente mediante la librería `Faker`, y posteriormente inician sesión.
- **Consulta y gestión del carrito y pedidos**: acciones frecuentes para visualizar y modificar el carrito y revisar pedidos realizados.

La configuración incluye la simulación de tiempos de espera entre acciones para aproximar el comportamiento real, y la elección aleatoria entre usuarios nuevos y existentes para autenticación.

Estos escenarios permiten generar una carga representativa y variada sobre la aplicación, facilitando la medición del rendimiento en condiciones próximas al uso real.

4.2. Archivos para las pruebas de carga

4.2.1. Archivo `load_test.py`

Este archivo define el comportamiento simulado de usuarios en la aplicación usando Locust. Se incluyen flujos de navegación, autenticación, compra y gestión de carrito para probar el rendimiento de la web.

```
1 from locust import HttpUser, task, between, constant
2 from faker import Faker
3 import random
4
5 fake = Faker()
```

```python

class QuickstartUser(HttpUser):
    wait_time = between(1, 2)
    host = "http://localhost:5000"

    def __init__(self, parent):
        super().__init__(parent)
        self.productos = []
        self.auth_token = None

    def on_start(self):
        # Obtener lista de productos disponibles
        response = self.client.get("/api/productos")
        self.productos = response.json()

        # Autenticar o registrar usuario
        if random.random() < 0.7:
            self.login_existing_user()
        else:
            self.register_new_user()

    def login_existing_user(self):
        # Usuario de prueba creado en app.py
        response = self.client.post("/login", {
            "email": "demo@test.com",
            "password": "demo123"
        })
        if response.status_code != 200:
            self.logout()

    def register_new_user(self):
        user_data = {
            "nombre": fake.name(),
            "email": fake.email(),
            "password": fake.password()
        }
        self.client.post("/registro", data=user_data)
        self.login(user_data["email"], user_data["password"
            ])

    def login(self, email, password):
        response = self.client.post("/login", {
            "email": email,
            "password": password
        })
        return response.status_code == 200

    @task(5)
    def browse_products(self):
        self.client.get("/")
```

```
55          self.client.get("/productos")
56          self.client.get("/api/productos")
57
58      @task(3)
59      def full_purchase_flow(self):
60          if not self.productos:
61              return
62
63          # Seleccionar producto aleatorio
64          producto = random.choice(self.productos)
65
66          # A adir al carrito
67          self.client.get(f"/carrito/agregar/{producto['id']}"
                )
68
69          # Ver carrito
70          self.client.get("/carrito")
71
72          # Realizar pedido (solo el 50% de las veces)
73          if random.random() < 0.5:
74              self.client.get("/pedido/realizar")
75              self.client.get("/pedidos")
76
77      @task(2)
78      def user_registration_flow(self):
79          user_data = {
80              "nombre": fake.name(),
81              "email": fake.email(),
82              "password": fake.password()
83          }
84          self.client.post("/registro", data=user_data)
85          self.login(user_data["email"], user_data["password"
                ])
86
87      @task(1)
88      def view_orders(self):
89          self.client.get("/pedidos")
90
91      @task(1)
92      def manage_cart(self):
93          self.client.get("/carrito")
94          if self.productos and random.random() < 0.3:
95              producto = random.choice(self.productos)
96              self.client.get(f"/carrito/agregar/{producto['id
                    ']}")
```

Listing 7: Simulación de usuarios concurrentes con Locust

4.3. Resultados

Se realizaron varias pruebas de carga para ver como un determinado número de usuarios afectaba el rendimiento del sistema.

4.3.1. Prueba 1

La configuración para esta prueba es:

- users: 20
- rampup: 5
- advanced options: 5m

Con una carga inicial de 20 usuarios distribuidos en 5 minutos, la aplicación responde con gran estabilidad. El número de usuarios se incrementa progresivamente como estaba previsto.

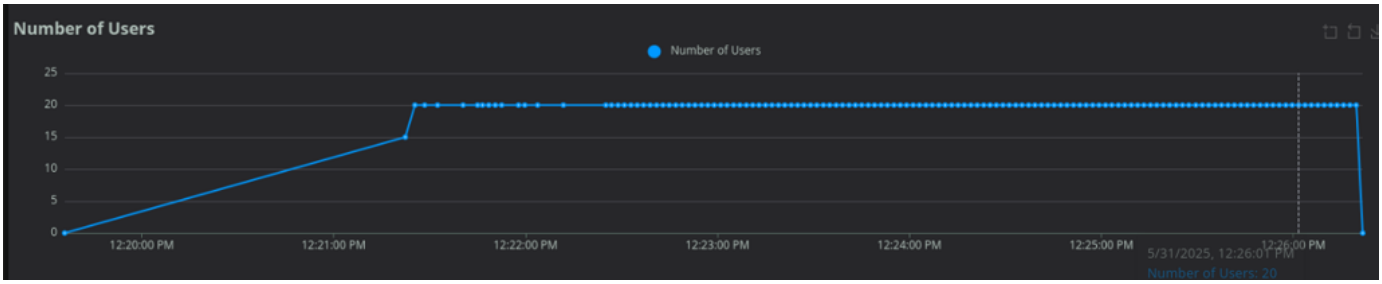


Figura 1: Gráfica del número de usuarios

Los tiempos de respuesta se mantienen por debajo de 200 ms en casi todas las peticiones, lo cual es muy aceptable para aplicaciones web.

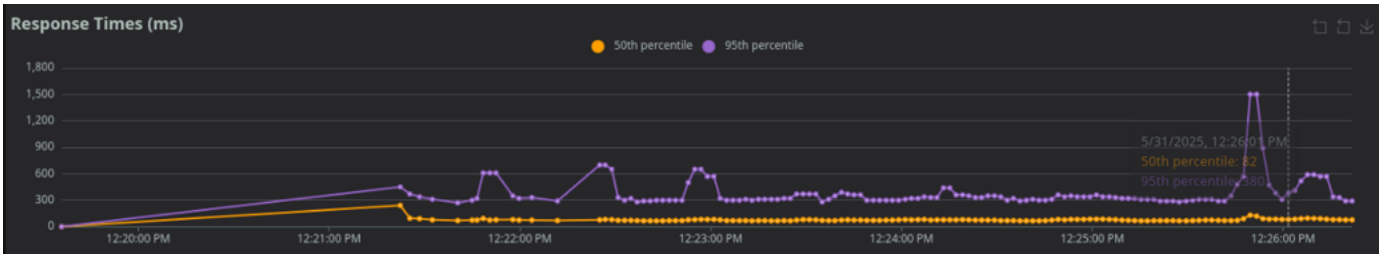


Figura 2: Gráfica de los tiempos de respuesta

La tasa de peticiones por segundo refleja un comportamiento fluido y sin cuellos de botella, alcanzando hasta 30 req/s sin saturación. Conclusión: rendimiento óptimo bajo carga baja.

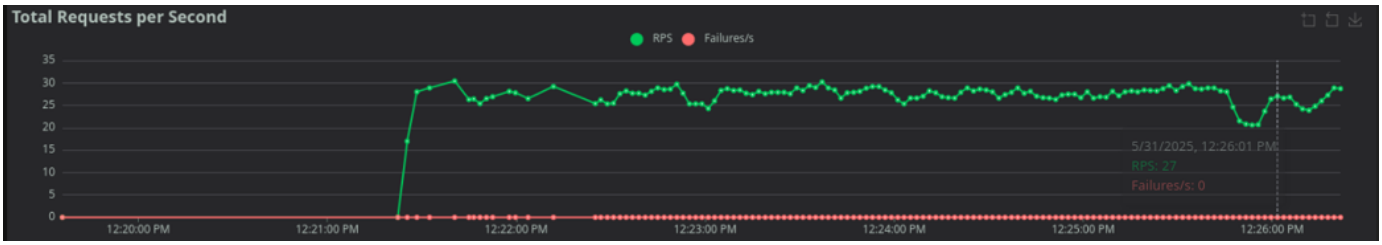


Figura 3: Gráfica de las peticiones por segundo

4.3.2. Prueba 2

La configuración para esta prueba es:

- users: 50
- rampup: 5
- advanced options: 5m

Aumentar a 50 usuarios mantiene un rendimiento aceptable.
Se observa una subida proporcional en las peticiones por segundo (hasta 60 req/s), sin picos de latencia significativos.

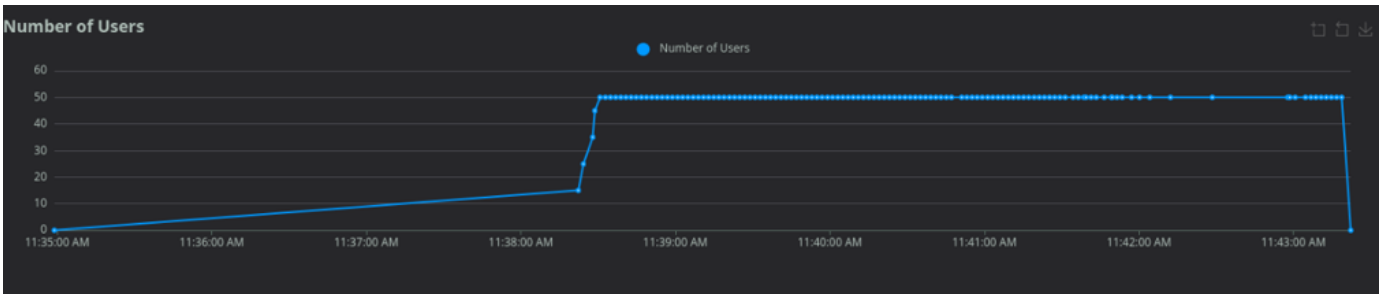


Figura 4: Gráfica del número de usuarios

La aplicación responde correctamente y de manera sostenida, manteniéndose en tiempos inferiores a 300 ms.

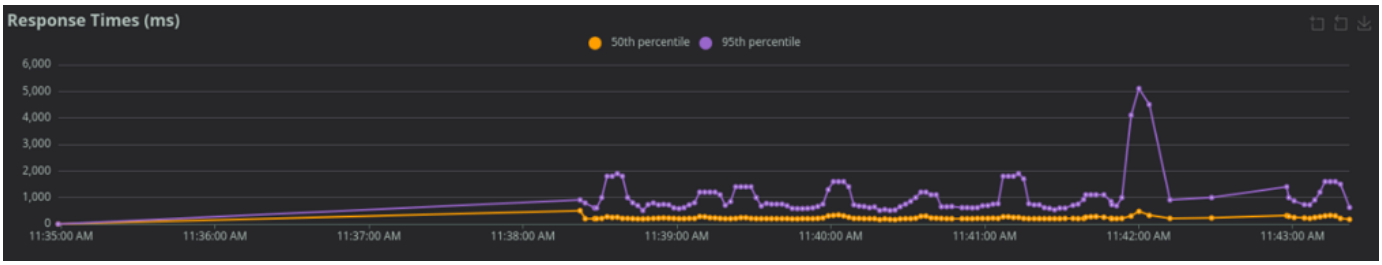


Figura 5: Gráfica de los tiempos de respuesta



Figura 6: Gráfica de las peticiones por segundo

Podemos ver que el sistema escala correctamente en cargas medias.

4.3.3. Prueba 3

La configuración para esta prueba es:

- users: 150
- rampup: 5
- advanced options: 5m

A partir de este punto se evidencia el límite de saturación del sistema:

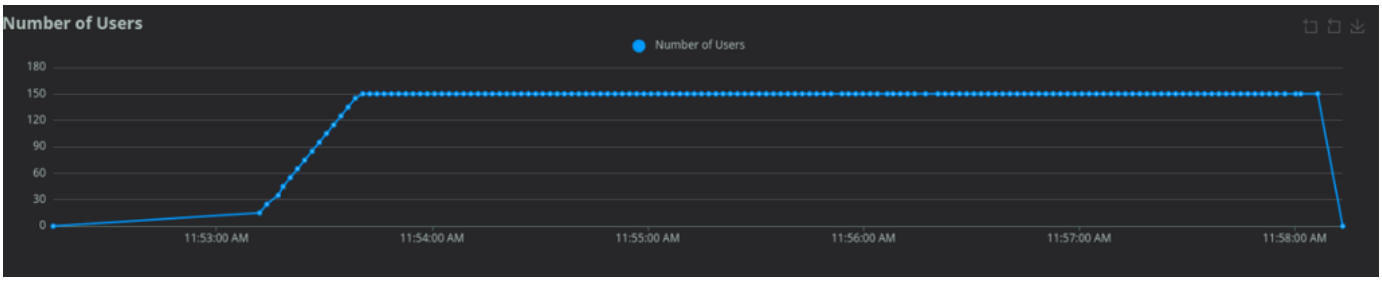


Figura 7: Gráfica del número de usuarios

Los tiempos de respuesta comienzan a oscilar y algunos picos superan los 500 ms, lo que implica cierta congestión del backend.

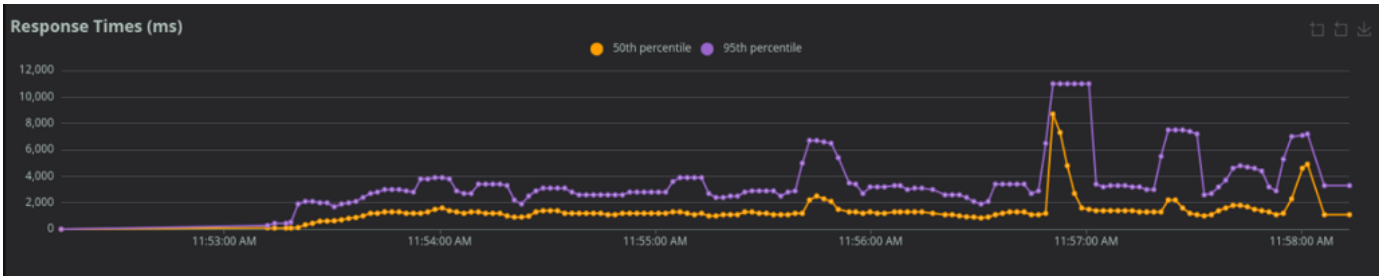


Figura 8: Gráfica de los tiempos de respuesta

La tasa de peticiones por segundo se estabiliza, pero ya no crece, lo cual indica cuello de botella.



Figura 9: Gráfica de las peticiones por segundo

El sistema empieza a degradarse ante alta concurrencia.

4.3.4. Prueba 4

La configuración para esta prueba es:

- users: 500
- rampup: 5
- advanced options: 5m

Este escenario satura completamente la aplicación.

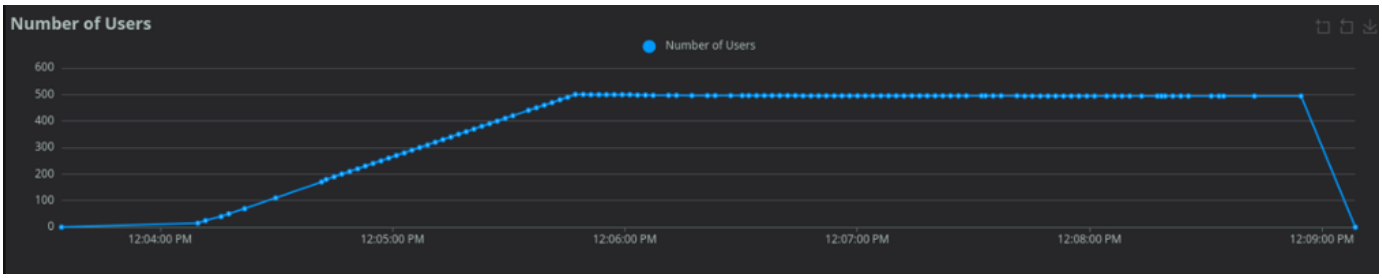


Figura 10: Gráfica del número de usuarios

Los tiempos de respuesta se disparan, llegando a varios segundos.

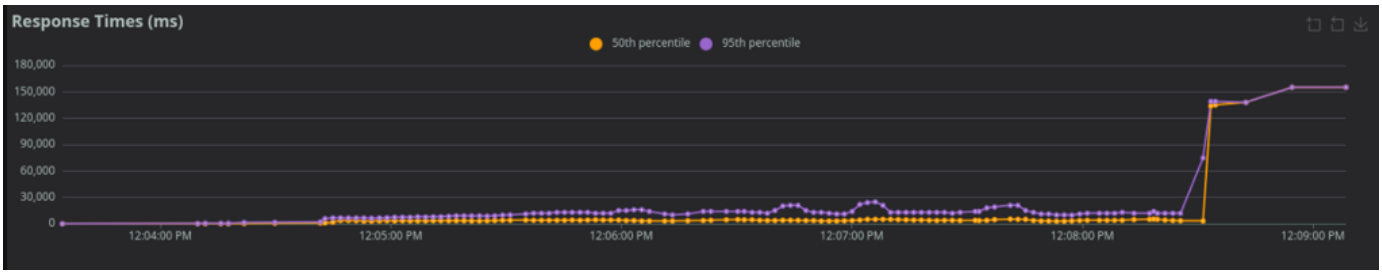


Figura 11: Gráfica de los tiempos de respuesta

Las peticiones por segundo caen notablemente, a pesar del aumento de usuarios, indicando que el sistema no es capaz de procesar eficientemente todas las solicitudes.



Figura 12: Gráfica de las peticiones por segundo

El sistema requiere optimización o escalabilidad horizontal para soportar cargas tan altas.

4.4. Conclusiones

Tras la realización de las pruebas Locust, hemos conseguido simular distintos flujos de usuarios de forma realista, además de evaluar la aplicación bajo dis-

tintas condiciones de carga. Vemos que la aplicación responde bien ante cargas bajas y medias, hasta 50 usuarios, con tiempos de respuesta razonables y una tasa de peticiones estable.

A partir de 150 usuarios se detecta síntomas de saturación, ya que vemos un incremento progresivo en los tiempos de respuesta y una reducción en la tasa de peticiones. Luego, con 500 usuarios, el sistema muestra ya una clara sobrecarga.

5. Monitorización con Grafana

En esta práctica se ha llevado a cabo la monitorización de diferentes aspectos del sistema, con el fin de obtener una visión detallada de su comportamiento durante las pruebas de carga. Más adelante en esta sección se desarrollan las medidas que se han registrado en que afectan en el rendimiento de la aplicación y de la máquina virtual.

5.1. Archivos relevantes para la monitorización

`grafana.sql`

Contiene las tablas para almacenar las métricas de sistema necesarias para la monitorización con Grafana.

La base de datos está compuesta por tres tablas principales relacionadas entre sí:

- `system_info`. Esta tabla contiene información estática o general del sistema, que rara vez cambia (como la capacidad total de memoria, disco o swap). Funciona como una tabla de configuración o referencia para identificar el entorno del sistema monitoreado.
- `system_metric`. Esta tabla registra métricas dinámicas del sistema a lo largo del tiempo, como el uso de CPU, memoria, disco, red, entre otros. Cada fila representa una "foto" del estado del sistema en un instante de tiempo.
- `process`. Esta tabla registra mt́ricas específicas de cada proceso individual del sistema, asociadas a una captura temporal (`system_metric`). Es una tabla de muchos registros por cada instante de tiempo (relación 1:N con `system_metric`).

```
1  -- Borrar tablas si ya existen
2  DROP TABLE IF EXISTS process;
3  DROP TABLE IF EXISTS system_metric;
4  DROP TABLE IF EXISTS system_info;
5
6  -- Crear tabla system_info
7  CREATE TABLE system_info (
8      id INT PRIMARY KEY AUTO_INCREMENT,
9      ram_total BIGINT NOT NULL,
10     swap_total BIGINT NOT NULL,
11     disk_total BIGINT NOT NULL
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
13
14 -- Crear tabla system_metric
15 CREATE TABLE system_metric (
16     id BIGINT PRIMARY KEY AUTO_INCREMENT,
17     system_info_id INT NOT NULL,
```

```
    timestamp DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP
        (6),
    cpu_load DOUBLE NOT NULL,
    cpu_freq DOUBLE NOT NULL,
    ram_used BIGINT NOT NULL,
    swap_used BIGINT NOT NULL,
    disk_used BIGINT NOT NULL,
    io_reads BIGINT NOT NULL,
    io_writes BIGINT NOT NULL,
    net_bytes_sent BIGINT NOT NULL,
    net_bytes_recv BIGINT NOT NULL,
    active_connections INT NOT NULL,
    process_count INT NOT NULL,
    CONSTRAINT fk_system_metric_system_info
        FOREIGN KEY (system_info_id)
        REFERENCES system_info(id)
        ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Crear tabla process
CREATE TABLE process (
    pid INT NOT NULL,
    system_metric_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    cpu_percent DOUBLE NOT NULL,
    mem_percent DOUBLE NOT NULL,
    PRIMARY KEY (pid, system_metric_id),
    CONSTRAINT fk_process_system_metric
        FOREIGN KEY (system_metric_id)
        REFERENCES system_metric(id)
        ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--  ndices  para rendimiento
CREATE INDEX idx_system_metric_timestamp ON system_metric(
    timestamp);
CREATE INDEX idx_system_metric_system_info_id ON
    system_metric(system_info_id);
CREATE INDEX idx_process_system_metric_id ON process(
    system_metric_id);
```

Listing 8: Contenido de grafana.sql

5.1.1. Archivo `main.py`

Este script ejecuta el proceso principal de monitorización del sistema. Establece la conexión con la base de datos, recoge métricas del sistema cada 60 segundos y las almacena de forma persistente. Se planteó obtener dichas métri-

cas cada 10 para no guardar tanta información en la base de datos dado que la memoria de nuestra máquina virutal tiene un porcentaje elevado de memoria utilizada.

```python
import time
from db import get_connection, insert_system_info,
    insert_system_metric, insert_processes
from metrics import get_system_info, get_metrics,
    get_processes

INTERVAL_SECONDS = 60

def main():
    conn = get_connection()
    print("Conectado a MariaDB")

    sys_info = get_system_info()
    system_info_id = insert_system_info(conn, **sys_info)

    print("Recogida de m tricas iniciada. Pulsa Ctrl+C para
        detener.")

    try:
        while True:
            data = get_metrics()
            metric_id = insert_system_metric(conn, data,
                system_info_id)
            processes = get_processes(limit=15)
            insert_processes(conn, processes, metric_id)
            print(f"M tricas registradas - ID {metric_id}")
            time.sleep(INTERVAL_SECONDS)
    except KeyboardInterrupt:
        print("\nFinalizando...")
    finally:
        conn.close()

if __name__ == '__main__':
    main()
```

Listing 9: Script principal de recogida de métricas

5.1.2. Archivo `metrics.py`

Este módulo contiene funciones para obtener las métricas del sistema mediante la biblioteca `psutil`. Se recogen datos de carga, memoria, disco, red y procesos activos.

```python
import psutil
import shutil
```

```python
import socket

def get_system_info():
    return {
        'ram_total': psutil.virtual_memory().total,
        'swap_total': psutil.swap_memory().total,
        'disk_total': shutil.disk_usage('/').total
    }

def get_metrics():
    net = psutil.net_io_counters()
    io = psutil.disk_io_counters()
    connections = len(psutil.net_connections(kind='inet'))

    return {
        'cpu_load': psutil.getloadavg()[0],  # 1 min avg
        'cpu_freq': psutil.cpu_freq().current,
        'ram_used': psutil.virtual_memory().used,
        'swap_used': psutil.swap_memory().used,
        'disk_used': shutil.disk_usage('/').used,
        'io_reads': io.read_count,
        'io_writes': io.write_count,
        'net_sent': net.bytes_sent,
        'net_recv': net.bytes_recv,
        'connections': connections,
        'proc_count': len(psutil.pids())
    }

def get_processes(limit=10):
    processes = []
    for proc in psutil.process_iter(['pid', 'name', '
        cpu_percent', 'memory_percent']):
        try:
            info = proc.info
            processes.append({
                'pid': info['pid'],
                'name': info['name'][:255],
                'cpu': info['cpu_percent'],
                'mem': info['memory_percent']
            })
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            continue
    # Ordenamos por CPU y cogemos los m s activos
    return sorted(processes, key=lambda p: p['cpu'], reverse
        =True)[:limit]
```

Listing 10: Funciones para obtener métricas del sistema

5.1.3. Archivo `db.py`

Aquí se definen las funciones para insertar la información obtenida en la base de datos MariaDB. Se separa la lógica de acceso a datos para facilitar el mantenimiento del código.

```
import mysql.connector
from config import DB_CONFIG

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)

def insert_system_info(conn, ram_total, swap_total,
    disk_total):
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO system_info (ram_total, swap_total,
            disk_total)
        VALUES (%s, %s, %s)
    """, (ram_total, swap_total, disk_total))
    conn.commit()
    return cursor.lastrowid

def insert_system_metric(conn, data, system_info_id):
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO system_metric (
            system_info_id, cpu_load, cpu_freq, ram_used,
                swap_used,
            disk_used, io_reads, io_writes, net_bytes_sent,
            net_bytes_recv, active_connections,
                process_count
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s
            , %s)
    """, (
        system_info_id,
        data['cpu_load'], data['cpu_freq'],
        data['ram_used'], data['swap_used'],
        data['disk_used'], data['io_reads'], data['io_writes
            '],
        data['net_sent'], data['net_recv'],
        data['connections'], data['proc_count']
    ))
    conn.commit()
    return cursor.lastrowid

def insert_processes(conn, processes, system_metric_id):
    cursor = conn.cursor()
    for p in processes:
        cursor.execute("""
```

```
39              INSERT INTO process (pid, system_metric_id, name
                    , cpu_percent, mem_percent)
40              VALUES (%s, %s, %s, %s, %s)
41          """, (p['pid'], system_metric_id, p['name'], p['cpu'
                ], p['mem']))
42      conn.commit()
```

Listing 11: Inserción de métricas en MariaDB

5.2. Métricas registradas

En esta seccion mostramos las graficas obtenidas mediante Grafana, a traves de las cuales vemos el impacto de la carga generada por Locust en los distintos recursos.

5.2.1. Carga instantánea de CPU

Prueba 1. 20 Usuarios en Locust

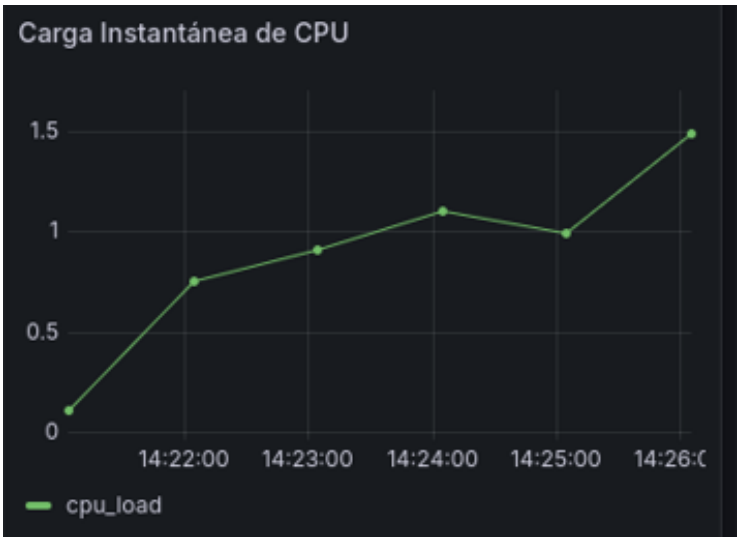


Figura 13: Gráfica de la carga instantánea de CPU. 50 usuarios

Prueba 2. 50 Usuarios en Locust

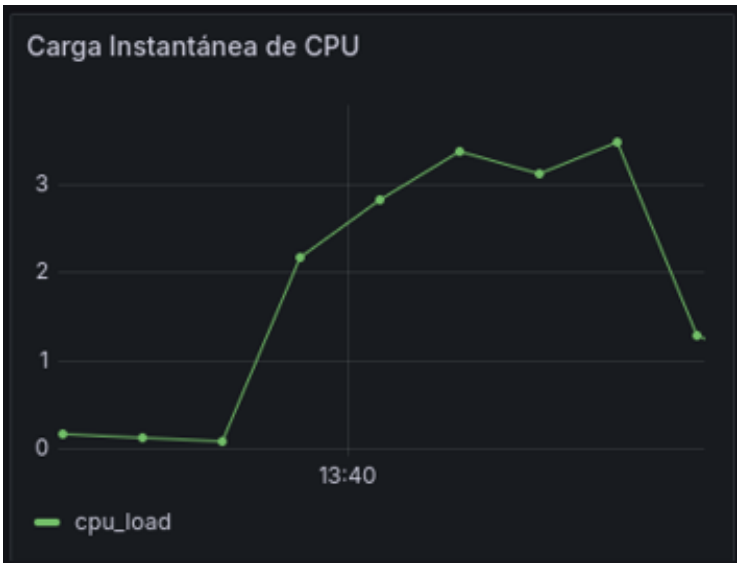


Figura 14: Gráfica de la carga instantánea de CPU. 100 usuarios

Prueba 3. 150 Usuarios en Locust

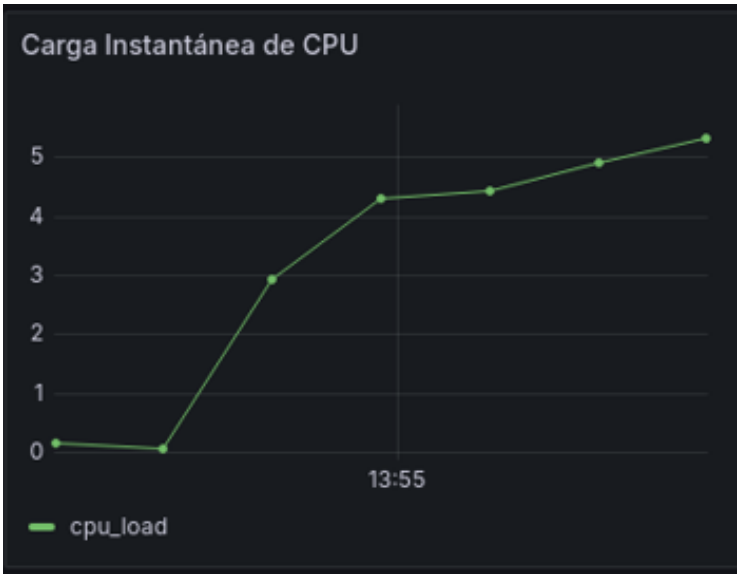


Figura 15: Gráfica de la carga instantánea de CPU. 150 usuarios

Prueba 4. 500 Usuarios en Locust5

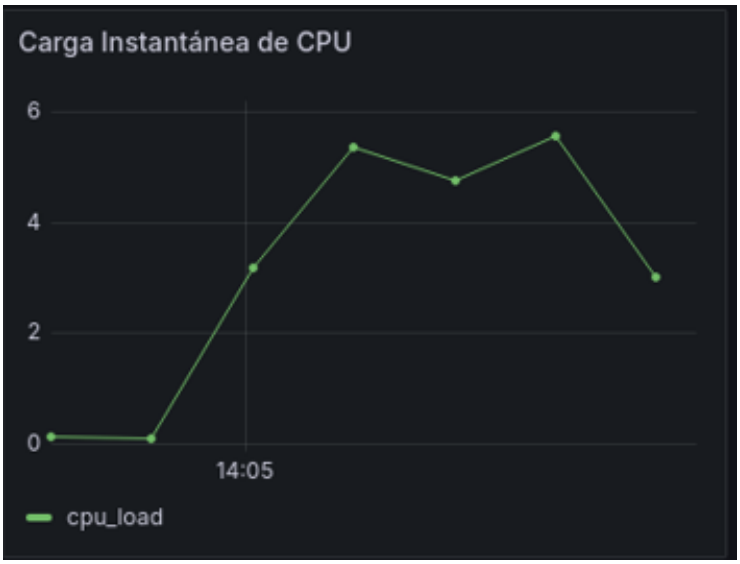


Figura 16: Gráfica de la carga instantánea de CPU. 500 usuarios

Durante las pruebas de carga, la métrica de carga de CPU mostró una clara tendencia ascendente conforme aumentaba el número de usuarios virtuales. En particular, a partir de los 150 usuarios concurrentes, la carga superó valores de 4–5, lo que indica una saturación del sistema en núcleos disponibles (recordando que la métrica refleja procesos en cola).

Esto sugiere que el servidor se encontraba por encima de su capacidad óptima, generando una ralentización general del sistema. Esta carga elevada puede afectar directamente a los tiempos de respuesta y al rendimiento percibido por los usuarios.

5.2.2. Frecuencia instantánea de CPU

Prueba 1. 20 Usuarios en Locust

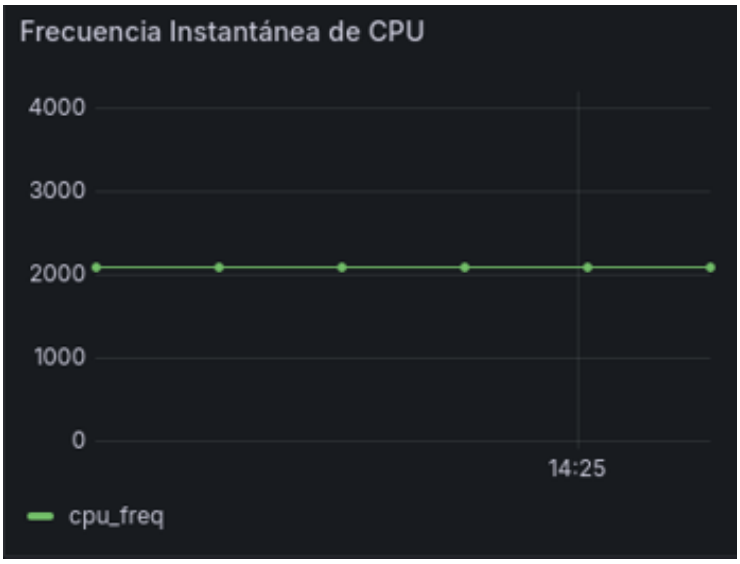


Figura 17: Gráfica de la frecuencia instantánea de CPU. 50 usuarios

Prueba 2. 50 Usuarios en Locust

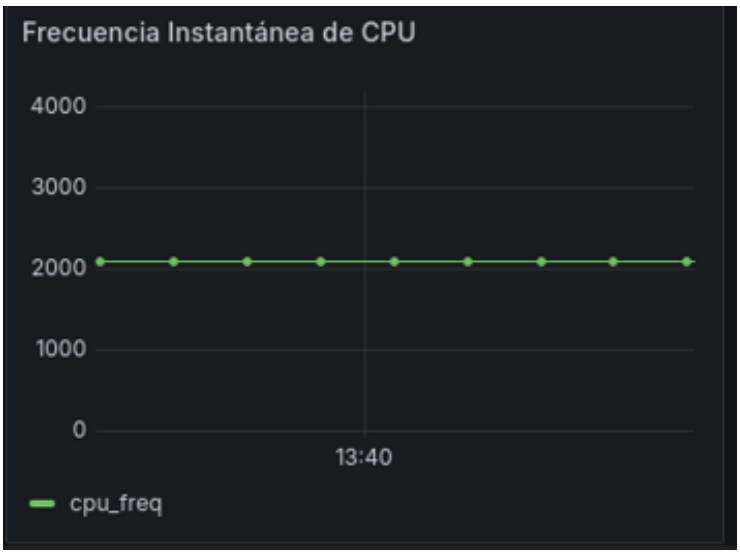


Figura 18: Gráfica de la frecuencia instantánea de CPU. 100 usuarios

Prueba 3. 150 Usuarios en Locust

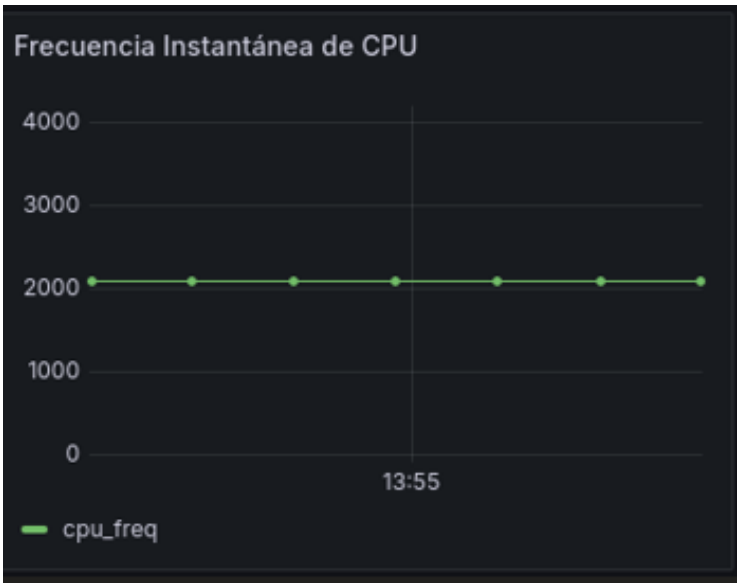


Figura 19: Gráfica de la frecuencia instantánea de CPU. 150 usuarios

Prueba 4. 500 Usuarios en Locust

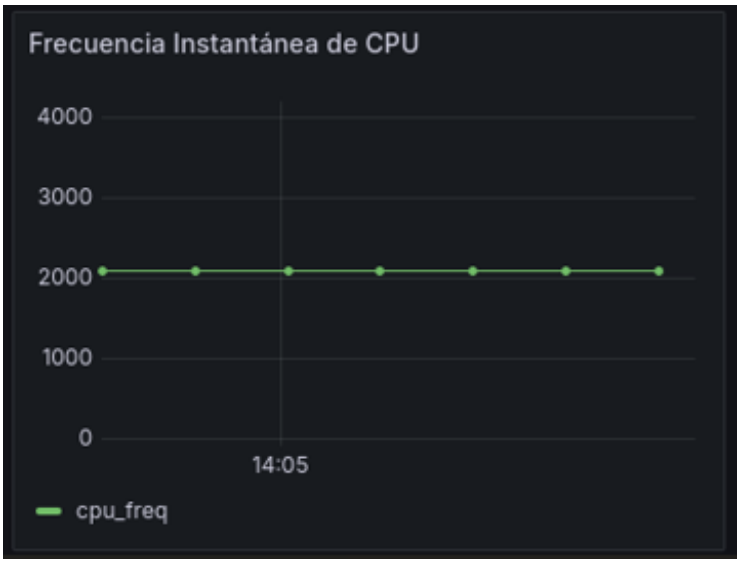


Figura 20: Gráfica de la frecuencia instantánea de CPU. 500 usuarios

La frecuencia del procesador se mantuvo estable durante toda la ejecución de las pruebas, sin variaciones significativas entre los distintos escenarios de carga. Este comportamiento indica que el sistema no realizó ajustes dinámicos en

la frecuencia del procesador, lo cual puede deberse a que el entorno de pruebas no activó mecanismos de escalado de frecuencia.

A diferencia de lo que podría ocurrir en sistemas con gestión energética activa, no se observaron ni picos ni caídas en la frecuencia, lo que implica un rendimiento predecible y constante a nivel de CPU en este entorno de prueba.

5.2.3. Uso de memoria RAM y memoria SWAP

Prueba 1. 20 Usuarios en Locust



Figura 21: Gráfica del uso de RAM. 50 usuarios



Figura 22: Gráfica del uso de memoria SWAP. 50 usuarios

Prueba 2. 50 Usuarios en Locust

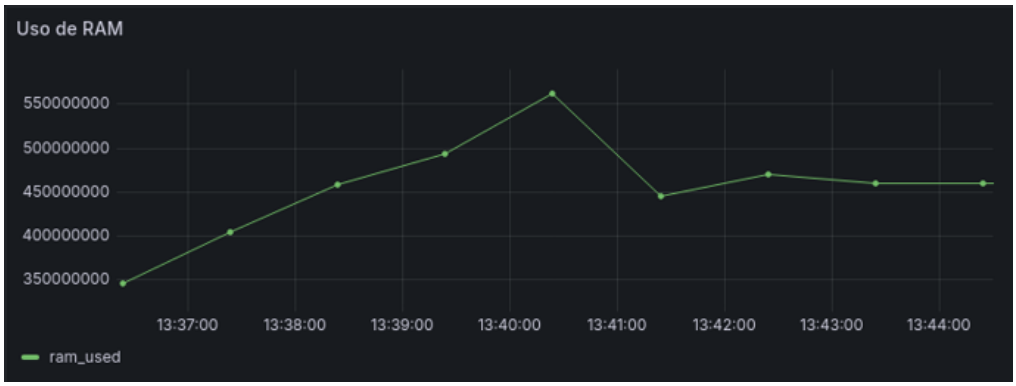


Figura 23: Gráfica del uso de RAM. 100 usuarios

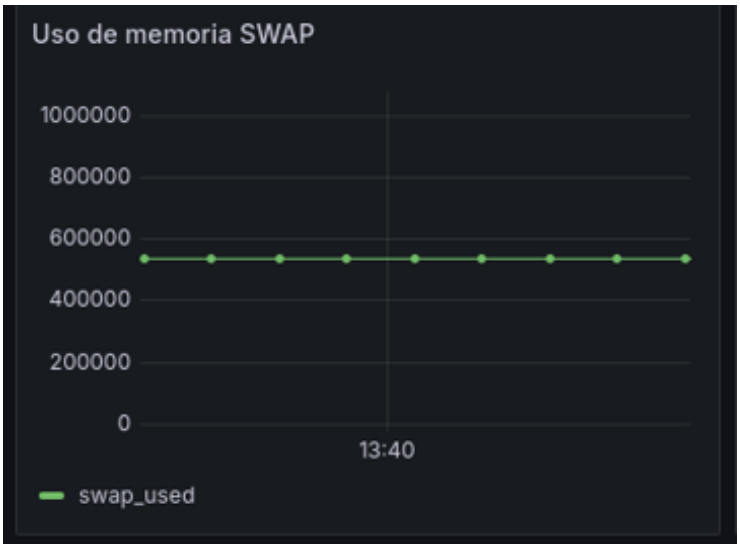


Figura 24: Gráfica del uso de memoria SWAP. 100 usuarios

Prueba 3. 150 Usuarios en Locust



Figura 25: Gráfica del uso de RAM. 150 usuarios

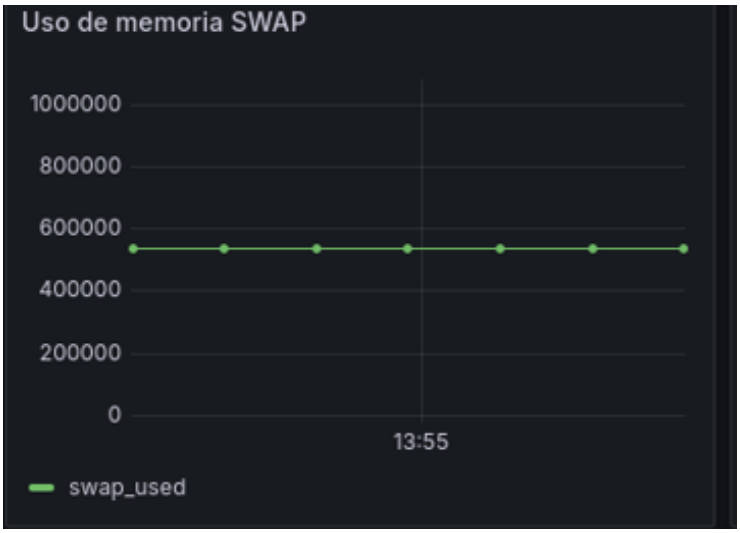


Figura 26: Gráfica del uso de memoria SWAP. 150 usuarios

Prueba 4. 500 Usuarios en Locust



Figura 27: Gráfica del uso de RAM. 500 usuarios

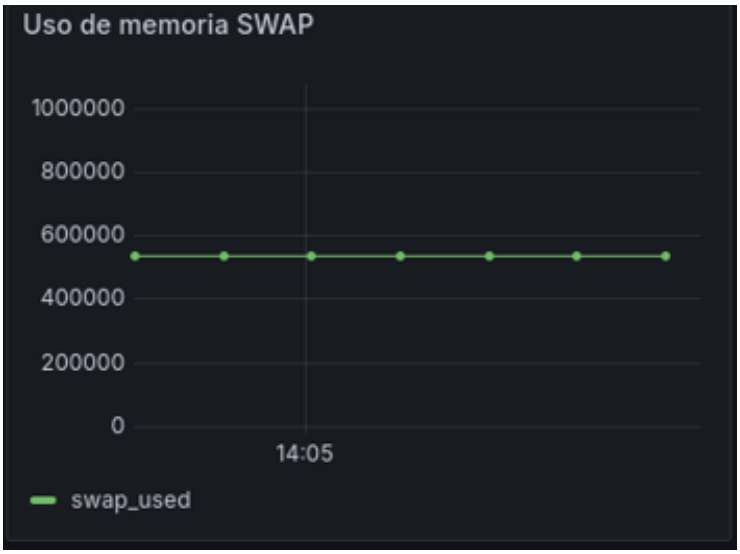


Figura 28: Gráfica del uso de memoria SWAP. 500 usuarios

El uso de memoria RAM aumentó progresivamente durante las simulaciones, alcanzando picos de más del 85 % de uso en los escenarios con mayor carga. Esto es coherente con el incremento en el número de procesos simultáneos (usuarios virtuales, sesiones activas, peticiones concurrentes...).

En cuanto al uso de SWAP, apenas se registraron accesos y es constante, lo cual es un signo positivo: el sistema fue capaz de mantener la carga en memoria física sin recurrir al espacio de intercambio en disco o con un uso excesivo, que tiene tiempos de acceso mucho más elevados.

5.2.4. Espacio en disco

Prueba 1. 20 Usuarios en Locust



Figura 29: Gráfica del uso de disco. 50 usuarios

Prueba 2. 50 Usuarios en Locust



Figura 30: Gráfica del uso de disco. 100 usuarios

Prueba 3. 150 Usuarios en Locust



Figura 31: Gráfica del uso de disco. 150 usuarios

Prueba 4. 500 Usuarios en Locust

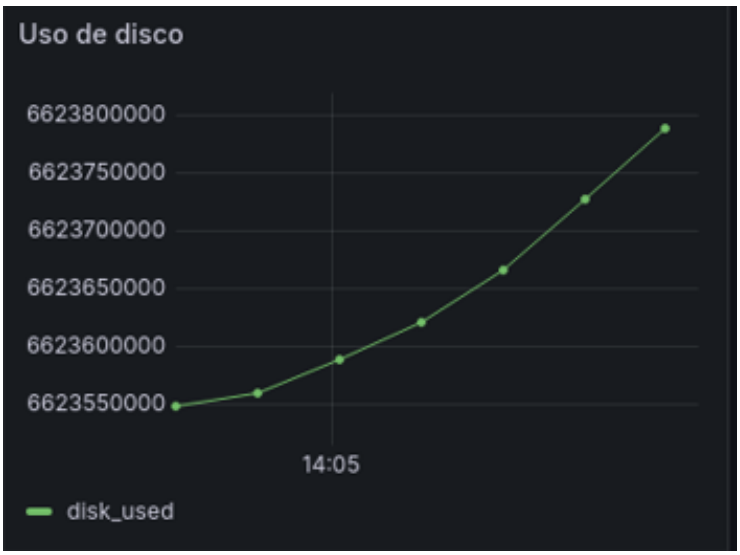


Figura 32: Gráfica del uso de disco. 500 usuarios

Como puede observarse, el consumo de disco refleja un aumento sostenido a medida que se incrementa la carga de usuarios. Esto es consistente con el

incremento de entradas en la base de datos generadas por los scripts de monitorización, en particular el número de procesos registrados. A pesar de ello, el sistema mostró estabilidad y nunca alcanzó valores de uso crítico. Es importante señalar que algunas pruebas presentan un aumento más gradual en el consumo de disco, mientras que otras siguen una tendencia más pronunciada, aunque sin llegar a un crecimiento abrupto.

5.2.5. Operaciones de entrada/salida (E/S)

Prueba 1. 20 Usuarios en Locust



Figura 33: Gráfica de las lecturas/escrituras de I/O. 50 usuarios

Prueba 2. 50 Usuarios en Locust



Figura 34: Gráfica de las lecturas/escrituras de I/O. 100 usuarios

Prueba 3. 150 Usuarios en Locust

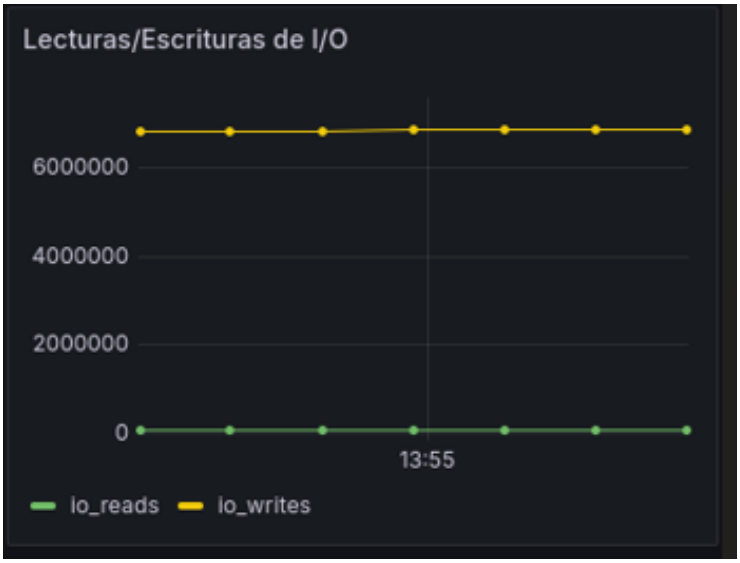


Figura 35: Gráfica de las lecturas/escrituras de I/O. 150 usuarios

Prueba 4. 500 Usuarios en Locust

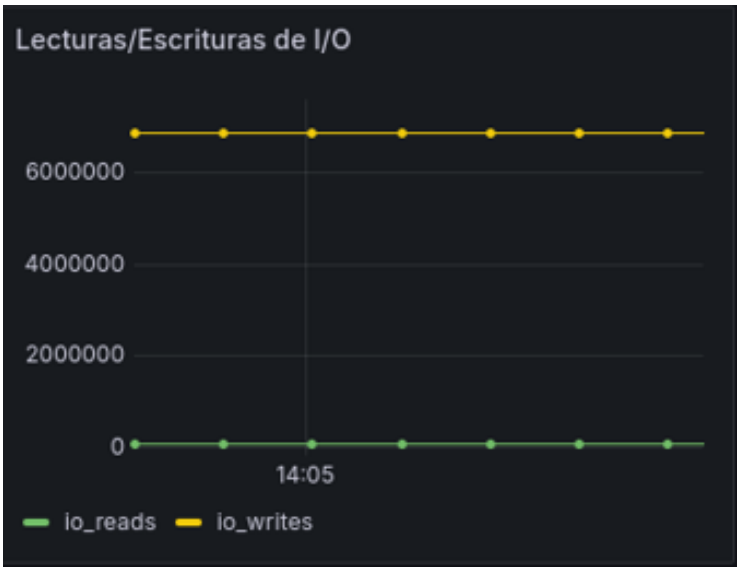


Figura 36: Gráfica de las lecturas/escrituras de I/O. 500 usuarios

La constancia en la actividad de E/S indica que, aunque el número de usuarios aumentó considerablemente, la cantidad de operaciones de lectura y escritura sobre el sistema de archivos no varió. Esto sugiere que la arquitectura de la aplicación está bien optimizada en cuanto al uso del disco, ya que las operaciones se realizan mayormente en memoria o mediante consultas eficientes a la base de datos.

Este resultado también reafirma que el subsistema de almacenamiento no fue un cuello de botella, incluso en escenarios de alta concurrencia.

5.2.6. Estadísticas de red: bytes enviados y recibidos

Prueba 1. 20 Usuarios en Locust

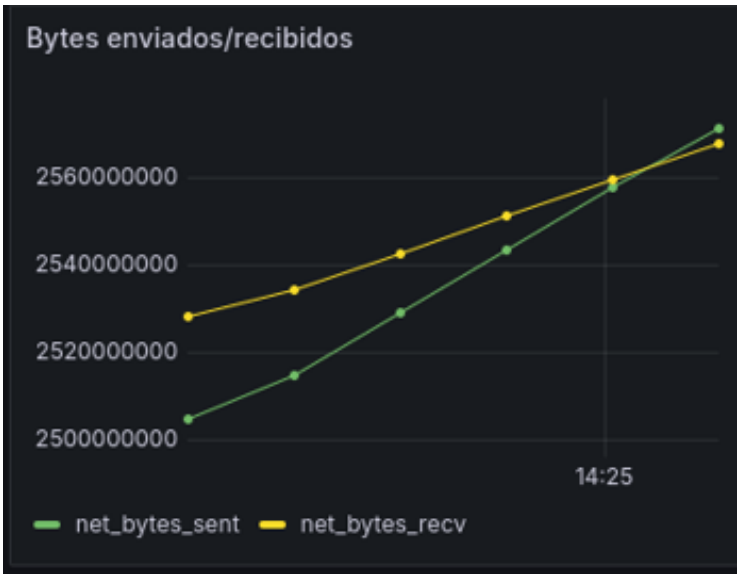


Figura 37: Primera Gráfica de los bytes enviados/recibidos. 50 usuarios

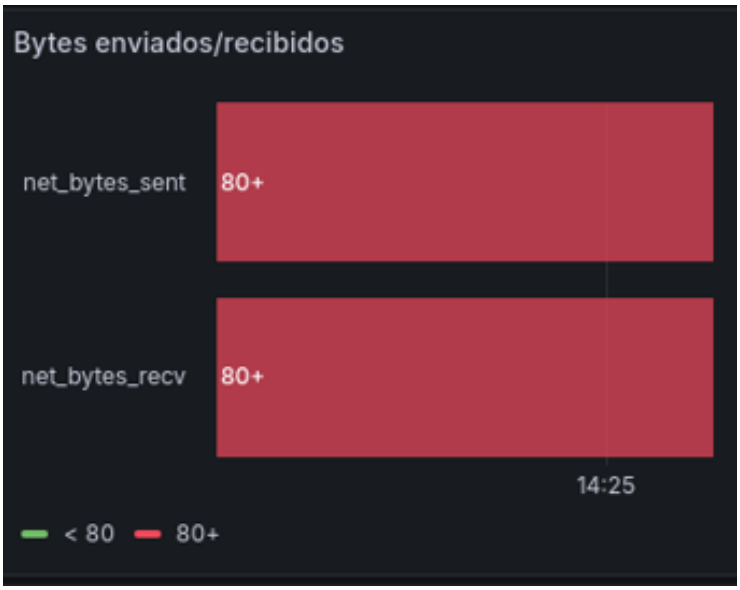


Figura 38: Segunda Gráfica de los bytes enviados/recibidos. 50 usuarios

Prueba 2. 50 Usuarios en Locust

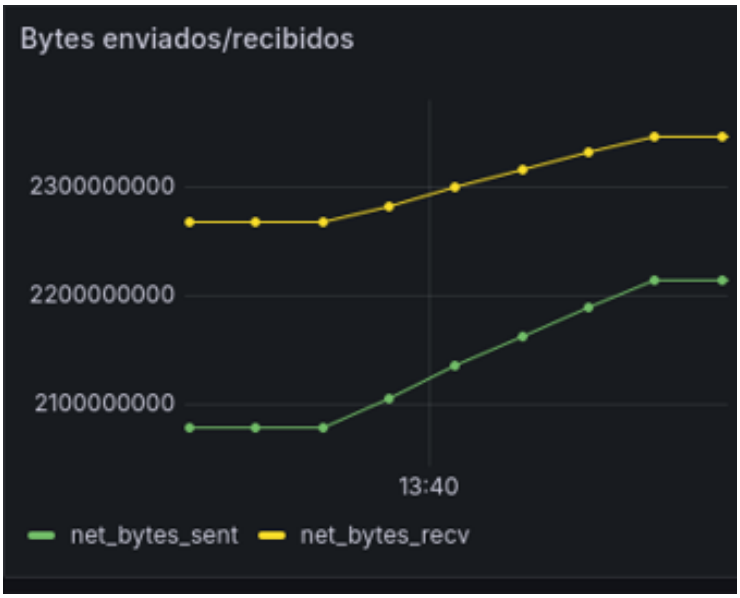


Figura 39: Primera Gráfica de los bytes enviados/recibidos. 100 usuarios



Figura 40: Segunda Gráfica de los bytes enviados/recibidos. 100 usuarios

Prueba 3. 150 Usuarios en Locust

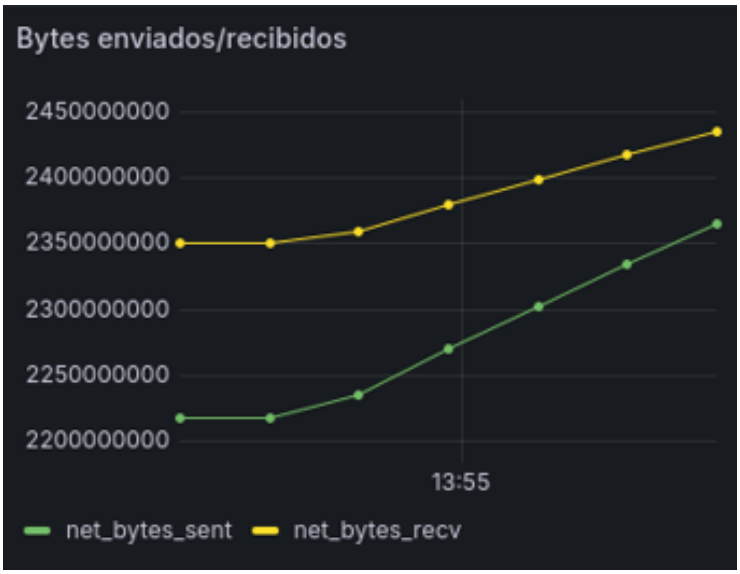


Figura 41: Primera Gráfica de los bytes enviados/recibidos. 150 usuarios



Figura 42: Segunda Gráfica de los bytes enviados/recibidos. 150 usuarios

Prueba 4. 500 Usuarios en Locust

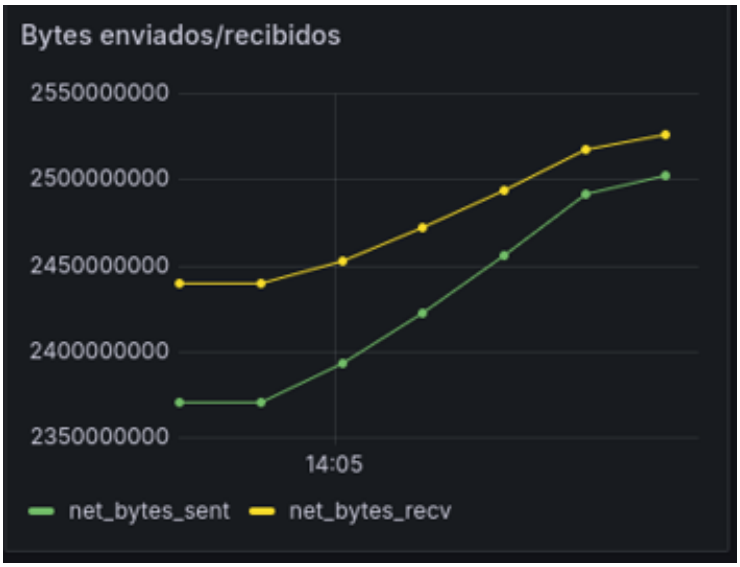


Figura 43: Primera Gráfica de los bytes enviados/recibidos. 500 usuarios

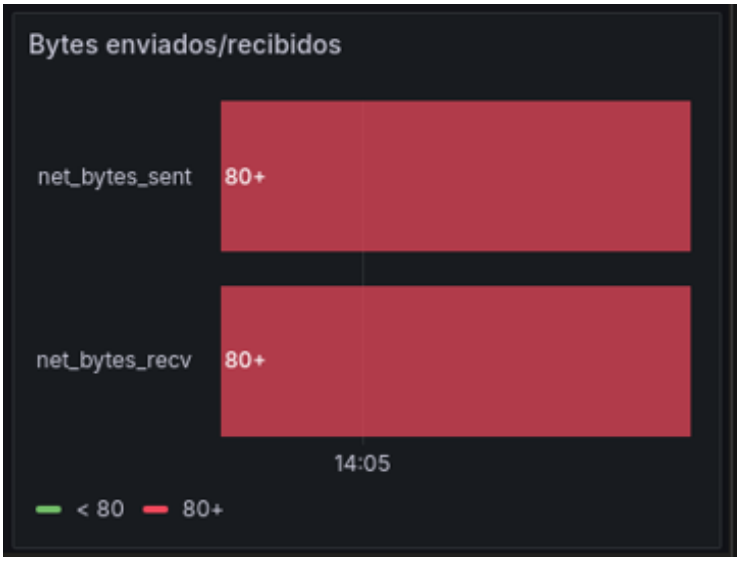


Figura 44: Segunda Gráfica de los bytes enviados/recibidos. 500 usuarios

Las métricas de red reflejan con claridad el aumento de tráfico conforme crece la carga de usuarios. Los picos en los datos enviados y recibidos coinciden con los momentos de mayor número de peticiones por segundo, según Locust. Esto demuestra una relación directa entre los accesos HTTP simulados y el tráfico de

red generado por la aplicación, con un comportamiento estable del adaptador de red, sin pérdidas ni caídas notables. La mayoría de las gráficas muestran un crecimiento que tiende a seguir una función logarítmica, con incrementos más pronunciados al inicio y una progresiva estabilización a medida que aumenta la carga

5.2.7. Conexiones de red activas

Prueba 1. 20 Usuarios en Locust

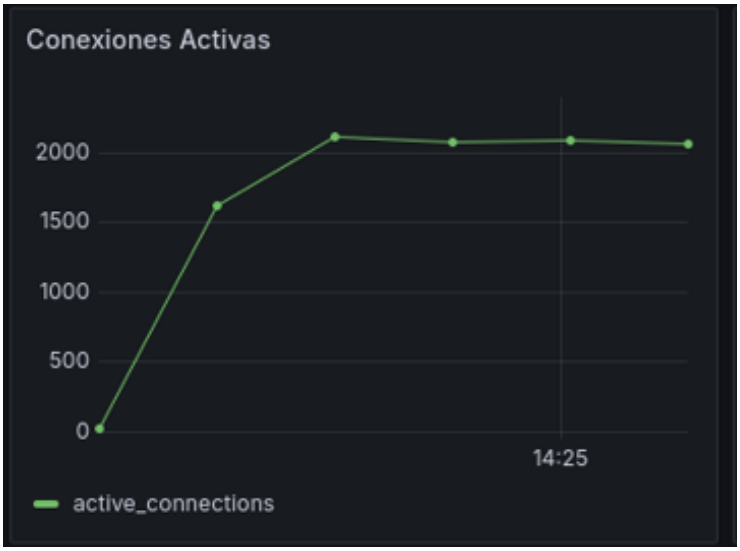


Figura 45: Gráfica de las conexiones activas. 50 usuarios

Prueba 2. 50 Usuarios en Locust

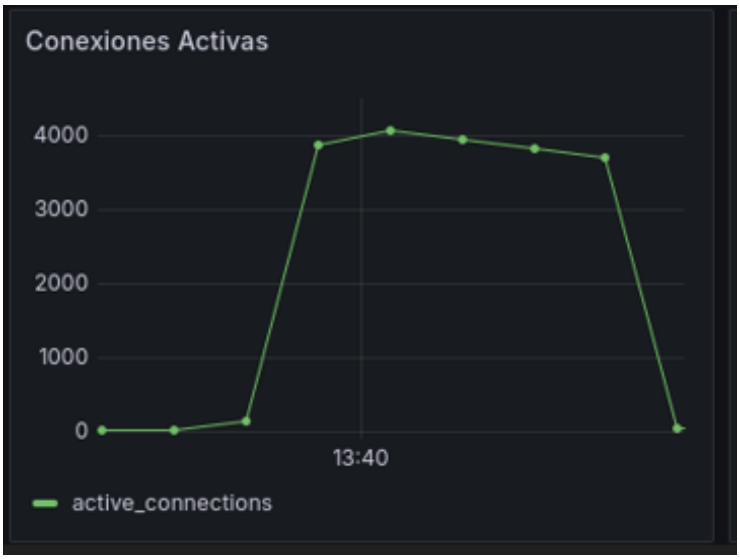


Figura 46: Gráfica de las conexiones activas. 100 usuarios

Prueba 3. 150 Ususarios en Locust

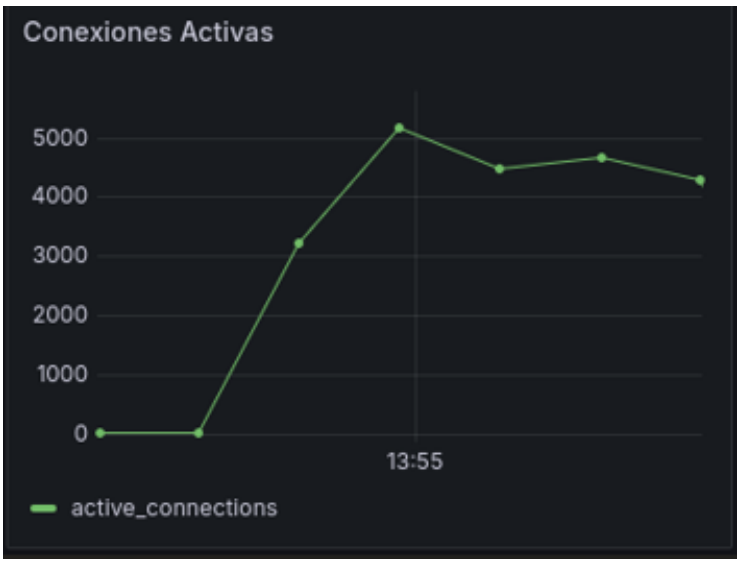


Figura 47: Gráfica de las conexiones activas. 150 usuarios

Prueba 4. 500 Usuarios en Locust

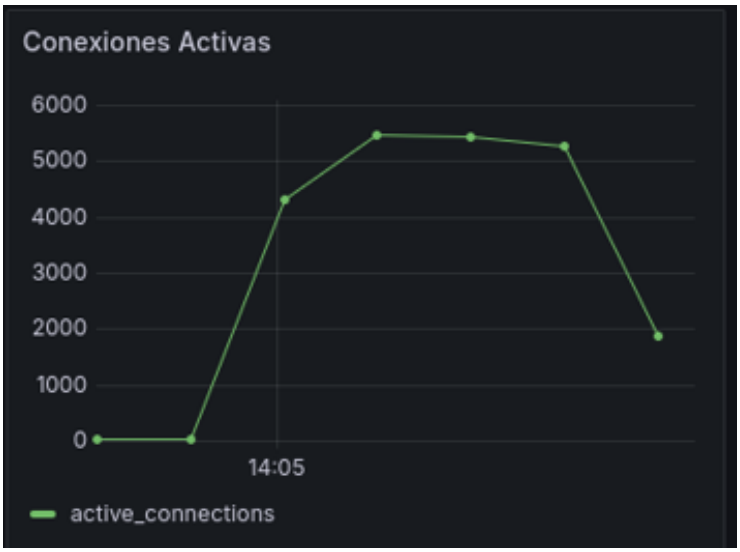


Figura 48: Gráfica de las conexiones activas. 500 usuarios

El número de conexiones activas creció significativamente a partir de los 100 usuarios, con picos superiores a 2000 conexiones simultáneas. Este parámetro es relevante ya que cada usuario genera múltiples conexiones (carga de imágenes, API, sesiones...).

Una elevada cantidad de conexiones abiertas simultáneamente puede afectar al rendimiento del servidor web, especialmente si no se tiene configurado un límite adecuado o se carece de mecanismos de control de concurrencia.

5.2.8. Número de procesos en ejecución

Prueba 1. 20 Usuarios en Locust



Figura 49: Gráfica del número de procesos. 50 usuarios

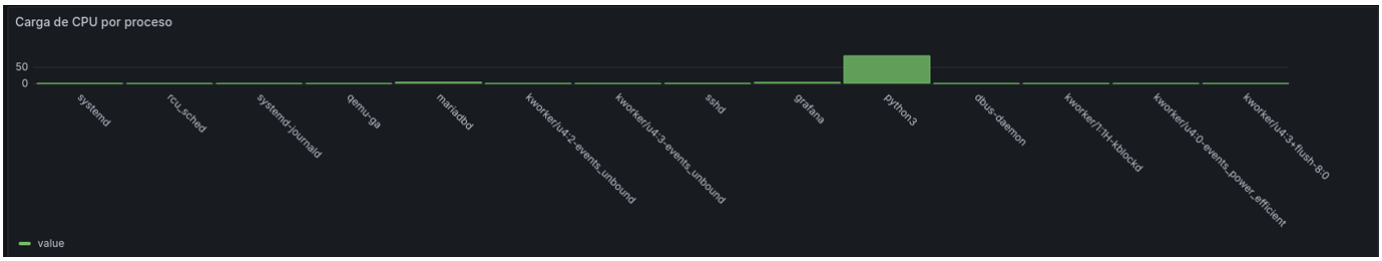


Figura 50: Gráfica de la carga de CPU por proceso. 50 usuarios

Prueba 2. 50 Usuarios en Locust



Figura 51: Gráfica del número de procesos. 100 usuarios

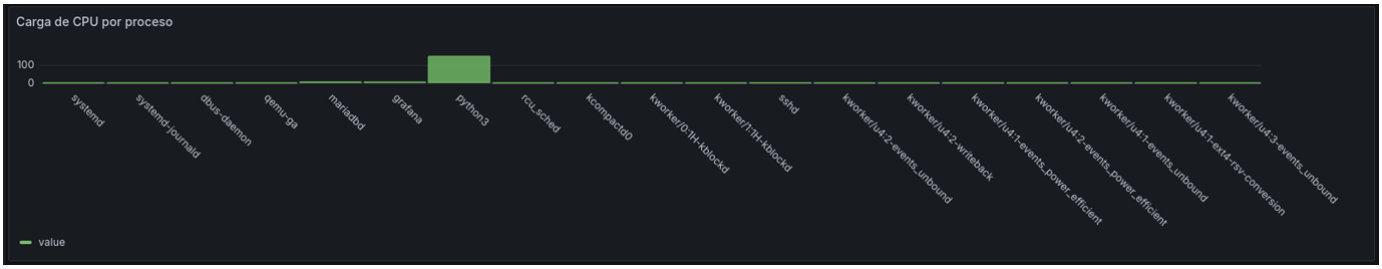


Figura 52: Gráfica de la carga de CPU por proceso. 100 usuarios

Prueba 3. 150 Usuarios en Locust



Figura 53: Gráfica del número de procesos. 150 usuarios

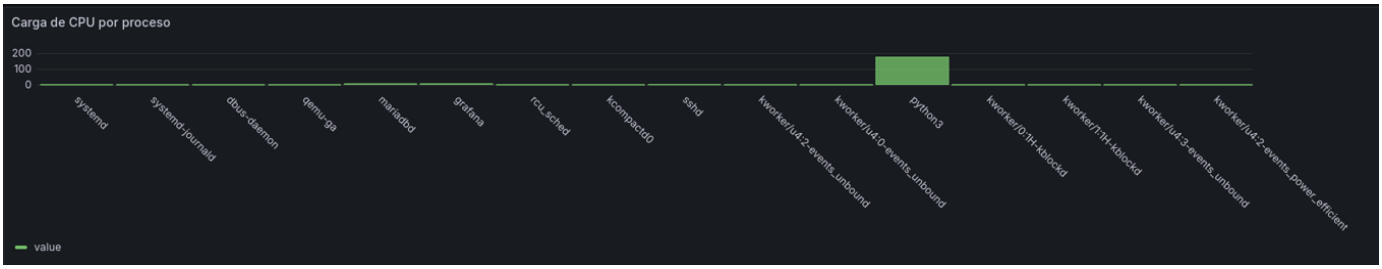


Figura 54: Gráfica de la carga de CPU por proceso. 150 usuarios

Prueba 4. 500 Usuarios en Locust



Figura 55: Gráfica del número de procesos. 500 usuarios



Figura 56: Gráfica de la carga de CPU por proceso. 500 usuarios

La cantidad de procesos se correlaciona directamente con el número de usuarios simulados, ya que cada usuario genera hilos o subprocesos para gestionar peticiones y sesiones. Este crecimiento puede generar una sobrecarga en el planificador del sistema operativo y aumentar el coste en cambios de contexto, lo que afecta el rendimiento general del sistema. Sin embargo, la carga de CPU por proceso se mantiene constante en 107 en todas las pruebas, lo que indica una distribución estable de recursos a nivel de proceso, sin incrementos significativos a medida que aumenta la carga de usuarios.

5.2.9. Lista detallada de los procesos en ejecución

Prueba 1. 20 Usuarios en Locust

Lista detallada de los procesos en ejecución

time	Process ID	Process Name	CPU Usage (%)	Memory Usage (%)
2025-05-31 14:26:04	1	systemd	0.300	0.638
2025-05-31 14:26:04	356	systemd-journald	0.100	12.4
2025-05-31 14:26:04	683	qemu-ga	0.100	0.0846
2025-05-31 14:26:04	94517	mariadbd	4.30	4.71
2025-05-31 14:26:04	542414	sshd	0.400	0.394
2025-05-31 14:26:04	542516	grafana	3.70	10.8
2025-05-31 14:26:04	542611	python3	0.100	1.36
2025-05-31 14:26:04	542622	python3	87.7	4.75
2025-05-31 14:25:04	675	dbus-daemon	0.100	0.241

Figura 57: Lista detallada de los procesos en ejecución. 50 usuarios

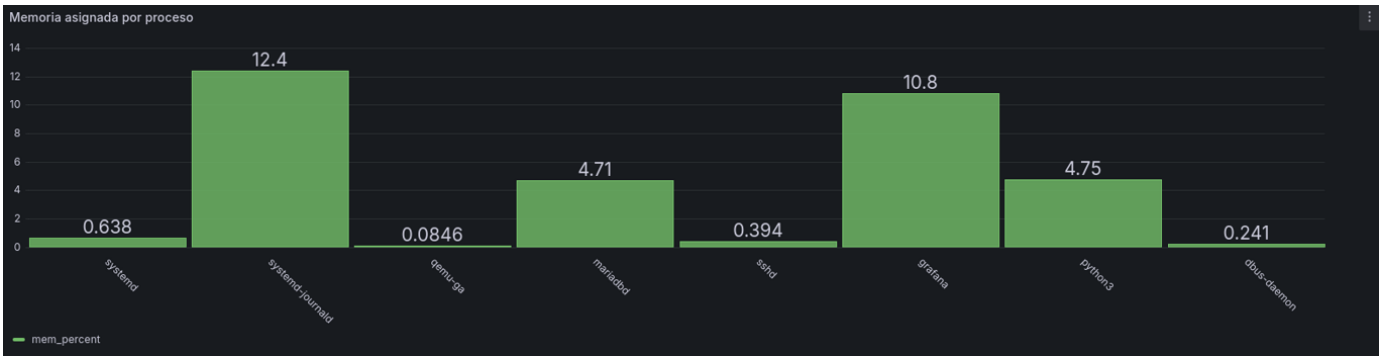


Figura 58: Segunda Gráfica de la memoria asignada por proceso. 50 usuarios

Prueba 2. 50 Usuarios en Locust

Lista detallada de los procesos en ejecución

time	Process ID	Process Name	CPU Usage (%)	Memory Usage (%)
2025-05-31 13:48:24	1	systemd	0.300	0.638
2025-05-31 13:48:24	356	systemd-journald	0.100	12.1
2025-05-31 13:48:24	675	dbus-daemon	0.100	0.241
2025-05-31 13:48:24	683	qemu-ga	0.100	0.0854
2025-05-31 13:48:24	94517	mariadbd	5.80	4.52
2025-05-31 13:48:24	439941	grafana	5.40	11.2
2025-05-31 13:48:24	472945	python3	149	9.63
2025-05-31 13:43:24	439663	sshd	0.800	0.490
2025-05-31 13:43:24	472918	python3	0.100	1.40

Figura 59: Lista detallada de los procesos en ejecución. 100 usuarios

Prueba 3. 150 Usuarios en Locust

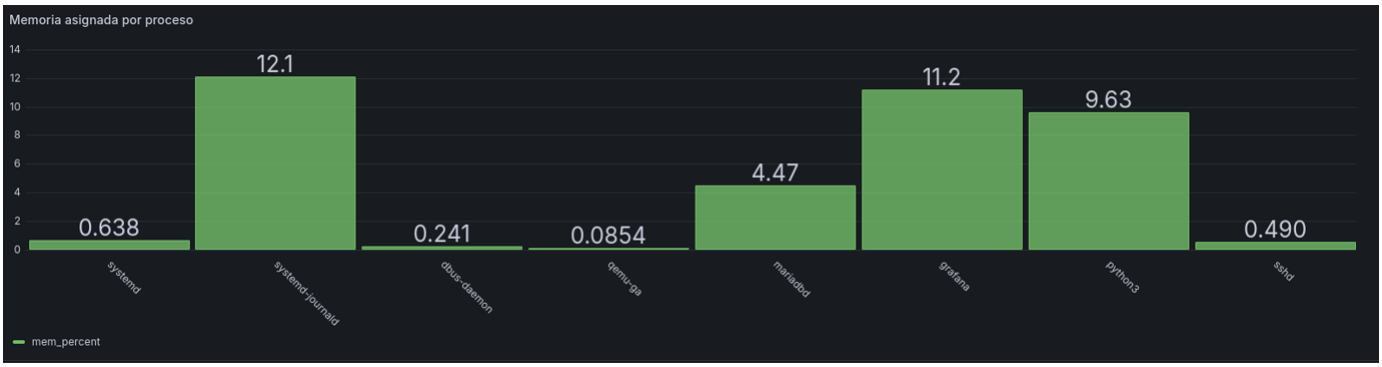


Figura 60: Segunda Gráfica de la memoria asignada por proceso. 100 usuarios

Lista detallada de los procesos en ejecución

time	Process ID	Process Name	CPU Usage (%)	Memory Usage (%)
2025-05-31 13:59:51	1	systemd	0.300	0.638
2025-05-31 13:59:51	356	systemd-journald	0.100	12.2
2025-05-31 13:59:51	675	dbus-daemon	0.100	0.241
2025-05-31 13:59:51	683	qemu-ga	0.100	0.0852
2025-05-31 13:59:51	94517	mariadbd	6	4.55
2025-05-31 13:59:51	439941	grafana	6.40	11.2
2025-05-31 13:57:51	439663	sshd	1.20	0.490
2025-05-31 13:57:51	493930	python3	0.100	1.42
2025-05-31 13:57:51	493943	python3	177	15.2

Figura 61: Lista detallada de los procesos en ejecución. 510 usuarios

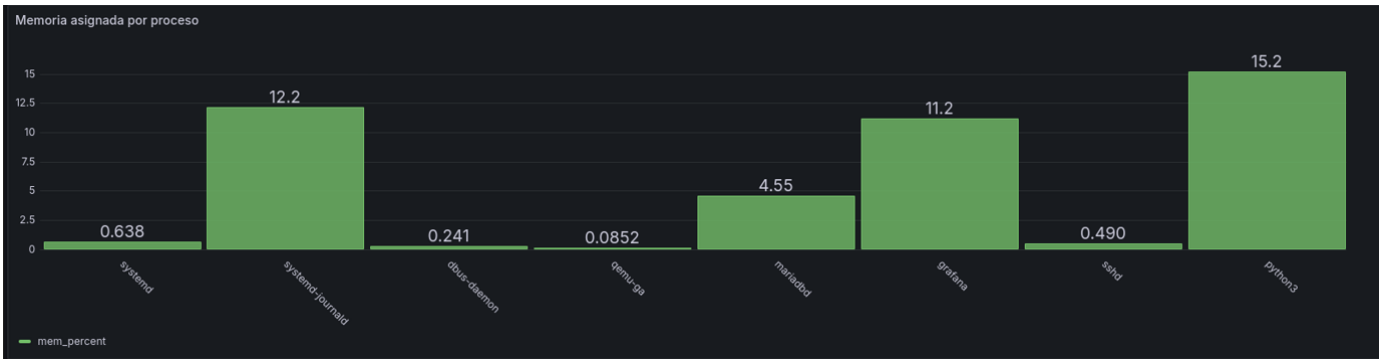


Figura 62: Segunda Gráfica de la memoria asignada por proceso. 150 usuarios

Prueba 4. 500 Usuarios en Locust

Lista detallada de los procesos en ejecución

time	Process ID	Process Name	CPU Usage (%)	Memory Usage (%)
2025-05-31 14:10:05.908	1	systemd	0.200	0.638
2025-05-31 14:10:05.908	356	systemd-journald	0.100	12.2
2025-05-31 14:10:05.908	675	dbus-daemon	0.100	0.241
2025-05-31 14:10:05.908	683	qemu-ga	0.100	0.0846
2025-05-31 14:10:05.908	94517	mariadbd	6.20	4.64
2025-05-31 14:10:05.908	439941	grafana	4.30	11.0
2025-05-31 14:09:05.803	439663	sshd	0.700	0.458
2025-05-31 14:09:05.803	518226	python3	0.100	1.46
2025-05-31 14:10:05.908	518241	python3	184	14.2

Figura 63: Lista detallada de los procesos en ejecución. 500 usuarios

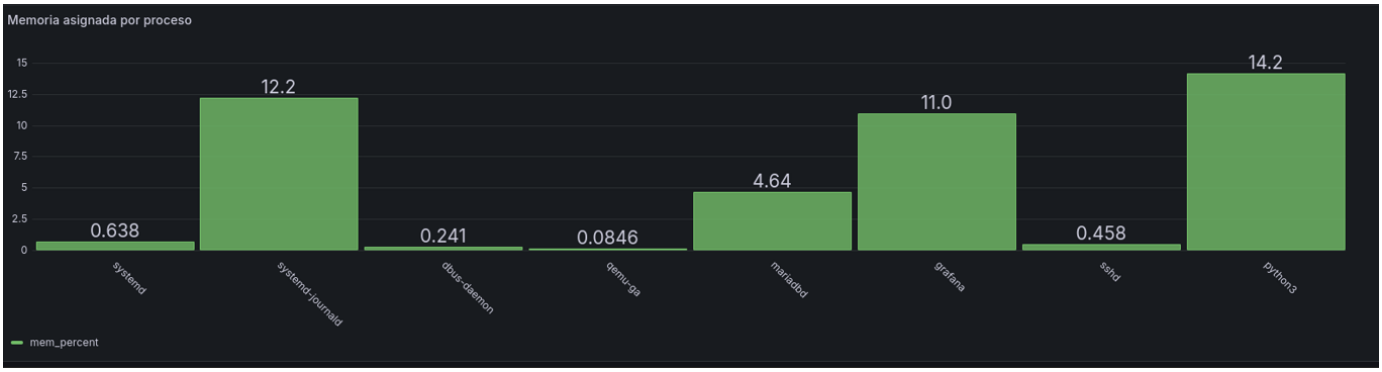


Figura 64: Segunda Gráfica de la memoria asignada por proceso. 500 usuarios

La lista de procesos recogida incluye aquellos con mayor consumo de CPU y memoria en cada instante. Esta información fue clave para identificar si existía algún componente concreto de la aplicación que estuviera provocando cuellos de botella.

Se identifican claramente los procesos de Python como los principales consumidores bajo carga, no es de extrañar, la recolección de datos, la página web y la simulación de carga de trabajo se han escrito en Python.

En general, se observó que los procesos más activos correspondían al servidor Flask y la base de datos, en proporción directa al escenario simulado. No se detectaron procesos anómalos o ajenos al entorno de pruebas consumiendo recursos.

5.3. Resultados

La monitorización con Grafana ha permitido obtener una visión completa y detallada del comportamiento del sistema bajo distintas condiciones de carga, revelando tanto fortalezas como posibles cuellos de botella.

En cuanto a la CPU, se ha observado un uso sostenido y creciente a medida que aumentaba el número de usuarios concurrentes. A partir de cierto umbral (150 usuarios), se alcanzaron niveles de carga cercanos o superiores al número de núcleos disponibles, lo que sugiere una posible saturación del sistema en escenarios de alta concurrencia.

Respecto al uso de memoria RAM, se detectó un incremento constante y significativo con cada prueba, lo cual es esperable debido al aumento de procesos y sesiones activas. Sin embargo, el uso de memoria SWAP se mantuvo constante y prácticamente nulo durante todas las pruebas, lo que indica que el sistema fue capaz de mantener la carga dentro de la memoria física, evitando el impacto negativo que supondría el acceso a disco.

El espacio en disco mostró un crecimiento progresivo debido al almacenamiento de métricas, pero sin alcanzar valores críticos en ningún momento. Este crecimiento fue gradual y estable, y no supuso un problema durante la ejecución de las pruebas.

Las operaciones de entrada/salida (E/S) se mantuvieron constantes incluso en pruebas con muchos usuarios, lo que sugiere que la aplicación está optimizada para minimizar la dependencia del sistema de archivos. Esto también implica que el subsistema de almacenamiento no se convirtió en un cuello de botella, ni siquiera bajo cargas elevadas.

Las estadísticas de red mostraron un incremento proporcional y predecible de tráfico (bytes enviados y recibidos), acorde con el número de usuarios simulados. No se detectaron pérdidas ni caídas significativas en la red. Asimismo, el número de conexiones activas creció de forma pronunciada, lo cual es coherente con el comportamiento esperado en entornos con muchos usuarios concurrentes.

Por último, tanto el número de procesos activos como los procesos más intensivos en consumo de CPU y memoria reflejaron una relación directa con la carga generada. Se identificaron claramente los procesos más críticos (servidor web, Locust, monitorización), sin presencia de procesos ajenos que distorsionaran el rendimiento.

En conjunto, la monitorización ha demostrado ser una herramienta esencial para correlacionar la carga generada por Locust con el impacto real sobre los recursos del sistema. Las métricas recopiladas permiten comprender de manera precisa el comportamiento de la aplicación y detectar las áreas que podrían requerir optimización o escalabilidad futura.

6. Conclusiones

A lo largo de esta práctica, hemos diseñado e implementado una aplicación web funcional, capaz de soportar distintos flujos de usuario (registro, navegación por productos, gestión de carrito y pedidos), y la hemos sometido a un riguroso proceso de evaluación de rendimiento mediante pruebas de carga con Locust y monitorización con Grafana.

Las pruebas con Locust nos permitieron simular escenarios realistas de uso concurrente, desde cargas bajas hasta situaciones de saturación. La aplicación mostró un rendimiento estable y eficiente bajo cargas moderadas (hasta 50 usuarios), manteniendo tiempos de respuesta bajos y una tasa de peticiones fluida. Sin embargo, a partir de 150 usuarios se empezaron a manifestar síntomas de congestión en el backend, y con 500 usuarios la sobrecarga fue evidente, con degradación del servicio y latencias elevadas.

Gracias a la monitorización con Grafana, pudimos correlacionar estos comportamientos con el consumo real de recursos del sistema. Se observó un uso creciente de CPU que alcanzó niveles críticos en pruebas exigentes, lo cual sugiere un potencial cuello de botella en escenarios de alta concurrencia. El uso de memoria RAM también aumentó de forma progresiva, pero la memoria SWAP se mantuvo constante y prácticamente nula, lo cual es positivo, ya que indica que la mayor parte de la actividad se mantuvo en memoria física, sin necesidad de acceder al disco.

El uso de disco y las operaciones de entrada/salida se mantuvieron estables, sin evidenciar saturación. Las métricas de red reflejaron el aumento proporcional del tráfico con el número de usuarios, sin pérdidas ni errores notables. También se detectó un crecimiento relevante en el número de conexiones activas y procesos del sistema, acorde con el incremento de la carga.

En conjunto, este trabajo ha proporcionado una visión integral del rendimiento de un sistema software bajo carga, combinando pruebas automatizadas, simulación realista de usuarios y recolección de métricas técnicas. Esta experiencia nos ha permitido identificar puntos fuertes de la aplicación y aspectos susceptibles de mejora, sentando las bases para futuras optimizaciones o procesos de escalado.